

Faculdade de Engenharia da Universidade do Porto



Computação Paralela e Distribuída

Relatório 1º Projeto

T14G14

João Lamas - up202208948@fe.up.pt

Miguel Mateus - up202206944@fe.up.pt

Pedro Fernandes - up202207987@fe.up.pt

Índice

Descrição do problema.....	3
Algoritmos.....	3
Simple Matrix Multiplication.....	3
Line Matrix Multiplication.....	4
Block Matrix Multiplication.....	4
Line Matrix Multiplication - Multi-core.....	5
Métricas de desempenho.....	6
Resultados e análise.....	6
Avaliação da performance single core.....	6
Avaliação da performance multi-core.....	7
Conclusões.....	7
Anexos.....	8

Descrição do problema

Com este trabalho pretende-se perceber o efeito da hierarquia da memória no desempenho do processador ao aceder a grandes quantidades de dados. Neste projeto utilizamos a multiplicação de matrizes que é uma operação fundamental em computação de alto desempenho, o que a torna altamente dependente da eficácia do uso da memória cache e RAM. Este relatório avalia diferentes abordagens para otimizar o desempenho da multiplicação de matrizes, considerando tanto a execução single-core como multi-core

Algoritmos

Foram implementados e avaliados quatro algoritmos de multiplicação de matrizes:

- Simple Matrix Multiplication
- Line Matrix Multiplication
- Block Matrix Multiplication
- Line Matrix Multiplication - Multi-core

Para os algoritmos de Simple and Line Matrix Multiplication foi-nos proposto fazermos a implementação destes mesmo algoritmos numa outra linguagem de programação, ao que a nossa escolha foi o uso de Java devido a ter uma fácil implementação e uma sintaxe parecida com C/C++.

Simple Matrix Multiplication

Neste algoritmo, cada elemento da matriz resultante é calculado multiplicando uma linha da primeira matriz por uma coluna da segunda matriz. Apesar da sua simplicidade, este método é altamente ineficiente para matrizes grandes devido ao acesso desorganizado à memória, resultando num elevado número de falhas de cache.

```
for(i=0; i<m_ar; i++)
{
    for( j=0; j<m_br; j++)
    {
        temp = 0;
        for( k=0; k<m_ar; k++)
        {
            temp += pha[i*m_ar+k] * phb[k*m_br+j];
        }
        phc[i*m_ar+j]=temp;
    }
}
```

Line Matrix Multiplication

Neste algoritmo cada elemento da primeira matriz é multiplicado pela linha correspondente da segunda matriz. Esta abordagem melhora a localidade espacial dos dados, reduzindo o número de falhas de cache em comparação com o método simples.

```
for(i=0; i<m_ar; i++) {
    for(k=0; k<m_br; k++) {
        double temp = pha[i*m_ar + k];
        for(j=0; j<m_br; j++) {
            phc[i*m_ar + j] += temp * phb[k*m_br + j];
        }
    }
}
```

Block Matrix Multiplication

Aqui, as matrizes são divididas em blocos menores, e a multiplicação é realizada em cada bloco separadamente. Esta técnica explora a localidade temporal e espacial, reduzindo ainda mais as falhas de cache. O tamanho do bloco é um parâmetro crítico que afeta diretamente o desempenho.

```
for(i=0; i<m_ar; i+=bkSize) {
    for(j=0; j<m_br; j+=bkSize) {
        for(k=0; k<m_ar; k+=bkSize) {
            for(ii=i; ii<i+bkSize && ii<m_ar; ii++) {
                for(jj=j; jj<j+bkSize && jj<m_br; jj++) {
                    double temp = 0.0;
                    for(kk=k; kk<k+bkSize && kk<m_ar; kk++) {
                        temp += pha[ii*m_ar + kk] * phb[kk*m_br + jj];
                    }
                    phc[ii*m_ar + jj] += temp;
                }
            }
        }
    }
}
```

Line Matrix Multiplication - Multi-core

Neste algoritmo temos duas versões paralelizadas do algoritmo de multiplicação linha por linha, utilizando OpenMP para distribuir o trabalho entre múltiplos núcleos de processamento.

Nesta primeira versão, o loop mais externo (i) é paralelizado, de forma a que cada linha seja apenas processada por uma thread.

```
#pragma omp parallel for
for(i=0; i<m_ar; i++) {
    for(k=0; k<m_br; k++) {
        double temp = pha[i*m_ar + k];
        for(j=0; j<m_br; j++) {
            phc[i*m_ar + j] += temp * phb[k*m_br + j];
        }
    }
}
```

Na segunda versão, apenas o loop mais interno (j) é paralelizado, o que pode causar problemas uma vez que várias threads podem tentar atualizar elementos da matriz final ao mesmo tempo, o que pode levar a uma diminuição da performance.

```
#pragma omp parallel
{
    for(i=0; i<m_ar; i++) {
        for(k=0; k<m_br; k++) {
            double temp = pha[i*m_ar + k];
            #pragma omp for
            for(j=0; j<m_br; j++) {
                phc[i*m_ar + j] += temp * phb[k*m_br + j];
            }
        }
    }
}
```

Métricas de desempenho

Para avaliar os nossos algoritmos foram utilizadas várias métricas de desempenho como :

- Tempo de Execução: Medido em segundos, reflete a eficiência do algoritmo em termos de velocidade.
- Falhas de Cache L1 e L2: Indicam o número de vezes que o processador não encontrou os dados necessários na cache, tendo que aceder à memória RAM. Quanto menor o número de falhas, melhor o uso da hierarquia de memória.
- Escalabilidade: Avaliação do desempenho em cenários single-core e multi-core, para entender como o algoritmo beneficia do paralelismo.

Resultados e análise

Avaliação da performance single core

No algoritmo Simple Matrix Multiplication, em C++ o tempo de execução aumenta quadraticamente com o tamanho da matriz, refletindo a complexidade $O(n^3)$ do algoritmo. As falhas de cache L1 e L2 são significativas, especialmente para matrizes maiores, devido ao acesso desorganizado à memória.

Já no algoritmo Line Matrix Multiplication em C++ apresenta uma melhoria significativa no tempo de execução e nas falhas de cache em comparação com o método simples. A localidade espacial dos dados é melhor aproveitada, resultando em menos acessos à memória RAM.

Já no algoritmo Block Matrix Multiplication em C++ o desempenho varia de acordo com o tamanho do bloco. Blocos menores (128x128) tendem a ser mais eficientes para matrizes menores, enquanto blocos maiores (512x512) são mais adequados para matrizes maiores. As falhas de cache são significativamente reduzidas em comparação com os métodos anteriores.

Em todos estes algoritmos o desempenho do algoritmo em Java é inferior ao de C++, com tempos de execução maiores para todas as dimensões de matriz. Isto pode ser atribuído à sobrecarga da Java Virtual Machine (JVM) e à gestão automática da memória.

Avaliação da performance multi-core

Na implementação do algoritmo Line Matrix Multiplication - Multi-core, foram criadas duas versões paralelas utilizando OpenMP, de forma a distribuir o trabalho entre vários núcleos de processamento. Ambas as implementações foram testadas com 4 threads.

A primeira implementação apresentou um ganho de desempenho significativo em relação à versão single-core, alcançando um speedup de cerca de 0,8x para matrizes menores e reduzindo gradualmente para aproximadamente 0,43x em matrizes maiores. Isto acontece devido ao aumento na carga de trabalho o que leva ao aumento do tempo gasto na sincronização dos threads. Para mais, a eficiência manteve-se entre 10% e 20%.

A segunda implementação teve um desempenho inferior, com um speedup consideravelmente menor, atingindo um máximo de 0,09x. A eficiência do paralelismo nesta versão manteve-se abaixo dos 3% em todos os testes. Esta versão é notavelmente inferior do que a primeira devido a estar a sincronizar as threads todas as interações de um ciclo interno.

Conclusões

Este trabalho ajudou-nos a perceber o efeito da hierarquia da memória no desempenho do processador ao aceder a grandes quantidades de dados. Neste projeto utilizamos a multiplicação de matrizes que é uma operação fundamental em computação de alto desempenho, o que a torna altamente dependente da eficácia do uso da memória cache e RAM. Para além disso, ganhamos mais conhecimento com as ferramentas utilizadas, linguagem Java e OpenMP, que é essencial para a paralelização de código. Por fim, ajudou-nos a entender melhor a importância de uma implementação eficiente.

Anexos

1.- Métricas execução Simple Matrix Multiplication - C++

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,1676652	246 096 874	2 094 344
1000x1000	1,0359422	1 140 491 032	16 425 143
1400x1400	3,419575	3 100 229 872	344 198 000
1800x1800	7,1487494	6 584 385 429	734 138 288
2200x2200	13,6117738	12 012 692 678	1 342 283 992
2600x2600	21,308263	19 821 793 451	2 244 648 034
3000x3000	35,1515912	30 439 697 941	3 537 913 356

2.- Métricas execução Simple Matrix Multiplication - Java

Tamanho Matriz	Tempo (s)
600x600	0,1811188
1000x1000	1,1048946
1400x1400	4,9237178
1800x1800	10,5259388
2200x2200	20,085021
2600x2600	33,5092876
3000x3000	53,4475462

3.- Métricas execução Line Matrix Multiplication - C++

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,0792418	27 644 605	668 712
1000x1000	0,5123396	128 772 698	3 140 711
1400x1400	1,384813	356 781 251	8 986 625
1800x1800	2,9177416	770 543 651	20 554 234
2200x2200	5,1876506	2 151 092 071	23 077 406
2600x2600	8,4975042	4 454 889 855	37 956 469
3000x3000	13,0323896	6 833 542 013	56 056 160
4096x4096	33,003688	17 405 531 409	168 244 927
6144x6144	111,291859	58 527 605 650	467 655 459
8192x8192	290,776201	138 614 445 509	1 120 772 055
10240x10240	730,191064	270 269 040 386	2 310 398 370

4.- Métricas execução Line Matrix Multiplication - Java

Tamanho Matriz	Tempo (s)
600x600	0,1117502
1000x1000	0,769838
1400x1400	1,7870536
1800x1800	3,4146538
2200x2200	6,4843158
2600x2600	10,9172928
3000x3000	16,9331926

5.- Métricas execução Block Matrix Multiplication - C++

5.1.- Métricas com Tamanho de Bloco = 128

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,0786196	38 777 500	430 717
1000x1000	0,3549174	177 667 801	1 950 896
1400x1400	1,1800876	483 655 113	4 373 109
1800x1800	3,0837562	1 020 963 473	9 498 138
2200x2200	5,0130806	1 851 219 238	14 296 173
2600x2600	9,0821642	3 058 305 743	24 317 201
3000x3000	11,1306482	4 637 714 685	36 886 883
4096x4096	30,478423	11 665 907 321	5 120 398 514
6144x6144	106,294255	39 238 038 794	17 945 038 884
8192x8192	869,486589	92 643 683 352	39 904 797 485
10240x10240	499,862844	181 479 469 414	82 145 807 145

5.2.- Métricas com Tamanho de Bloco = 256

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,0830426	34 439 867	6 129 085
1000x1000	0,3711602	154 106 991	35 344 010
1400x1400	1,4385776	427 078 599	87 860 050
1800x1800	2,3814404	893 629 048	209 030 418
2200x2200	5,6803378	1 611 403 550	385 430 703
2600x2600	8,519732	2 661 849 080	686 097 226
3000x3000	10,8882842	4 056 364 501	1 013 557 552
4096x4096	85,903788	10 211 325 071	2 534 142 639
6144x6144	103,769823	34 180 606 184	9 312 393 876
8192x8192	676,731733	81 238 358 111	20 352 747 671
10240x10240	530,886028	158 433 289 581	43 264 417 965

5.3.- Métricas com Tamanho de Bloco = 512

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,073681	32 178 340	3 350 802
1000x1000	0,3887136	141 781 844	21 192 103
1400x1400	1,3815052	390 506 911	61 294 663
1800x1800	2,7170018	833 882 076	129 420 970
2200x2200	5,8329868	1 498 695 065	261 802 478
2600x2600	7,8281472	2 480 049 841	372 913 014
3000x3000	10,90242875	3 762 203 271	593 315 048
4096x4096	61,965211	9 514 683 561	1 141 571 666
6144x6144	206,743539	31 905 654 658	3 798 430 140
8192x8192	488,000756	75 804 591 549	9 180 044 640
10240x10240	970,103086	147 924 440 469	17 433 448 312

6.- Métricas execução Block Matrix Multiplication - Java

6.1.- Métricas com Tamanho de Bloco = 128

Tamanho Matriz	Tempo (s)
600x600	0,176178
1000x1000	0,8187488
1400x1400	2,2425592
1800x1800	4,6882658
2200x2200	9,0439032
2600x2600	14,0107202
3000x3000	21,4431382

6.2.- Métricas com Tamanho de Bloco = 256

Tamanho Matriz	Tempo (s)
600x600	0,180345
1000x1000	0,8472618
1400x1400	2,261112
1800x1800	4,6680546
2200x2200	8,726274
2600x2600	14,2600226
3000x3000	21,9692734

6.3.- Métricas com Tamanho de Bloco = 512

Tamanho Matriz	Tempo (s)
600x600	0,192893
1000x1000	0,9440428
1400x1400	2,5442212
1800x1800	5,6006534
2200x2200	10,4782978
2600x2600	16,0338766
3000x3000	24,8517846

7.- Métricas execução Line Matrix Multiplication com OpenMP (Primeira implementação) a utilizar 4 Threads

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	0,098972	7 005 620	155 713
1000x1000	0,750375	32 525 203	828 451
1400x1400	2,593589	93 483 467	2 247 728
1800x1800	6,30831	207 388 141	5 109 173
2200x2200	11,747016	545 749 161	5 750 524
2600x2600	19,714086	1 110 946 837	9 300 252
3000x3000	30,141177	1 704 856 666	14 173 784

8.- Métricas execução Line Matrix Multiplication com OpenMP (Segunda implementação) a utilizar 4 Threads

Tamanho Matriz	Tempo (s)	L1 DCM	L2 DCM
600x600	1,1010786	35 413 681	5 433 264
1000x1000	5,3102468	174 951 121	25 229 722
1400x1400	16,5225028	559 405 693	97 984 810
1800x1800	34,19171	1 055 963 823	88 177 544
2200x2200	64,5515068	2 691 110 320	259 576 142
2600x2600	101,26927	4 624 701 428	229 609 904
3000x3000	175,4855394	7 028 315 446	261 114 934

9.- Estatísticas de execução Line Matrix Multiplication com OpenMP (Primeira implementação) a utilizar 4 Threads

Tamanho Matriz	Tempo (s)	MFlops	Speedup	Eficiência
600x600	0,098972	4364,870873	0,800651	20,02%
1000x1000	0,750375	2665,334	0,682779	17,07%
1400x1400	2,593589	2115,986766	0,533937	13,35%
1800x1800	6,30831	1848,989666	0,462524	11,56%
2200x2200	11,747016	1812,885928	0,441614	11,04%
2600x2600	19,714086	1783,090527	0,431037	10,78%
3000x3000	30,141177	1791,569055	0,432378	10,81%

10.- Estatísticas de execução Line Matrix Multiplication com OpenMP
(Segunda implementação) a utilizar 4 Threads

Tamanho Matriz	Tempo (s)	MFlops	Speedup	Eficiência
600x600	1,101079	392,34242	0,071968	1,80%
1000x1000	5,310247	376,630315	0,096481	2,41%
1400x1400	16,522503	332,153064	0,083814	2,10%
1800x1800	34,19171	341,135322	0,085335	2,13%
2200x2200	64,551507	329,907093	0,080365	2,01%
2600x2600	101,26927	347,114184	0,08391	2,10%
3000x3000	175,485539	307,717663	0,074265	1,86%